

# **Competitive Programming**

## **Section 67: Persistence, Rollback, And Time**

Competitive Programming Course

# Table of contents

|    |                                                             |    |
|----|-------------------------------------------------------------|----|
| 1  | Two ways to preserve the past                               | 3  |
| 2  | Path-copying persistence                                    | 4  |
| 3  | Persistent lazy propagation                                 | 5  |
| 4  | Rollback DSU                                                | 6  |
| 5  | Segment tree over time                                      | 7  |
| 6  | Cost accounting                                             | 8  |
| 7  | Worked trace: branching versions are a tree, not a timeline | 9  |
| 8  | Worked trace: why edge lifetimes are half-open              | 10 |
| 9  | Choosing persistence or rollback                            | 11 |
| 10 | C++ and Python implementation notes                         | 12 |
| 11 | Proof habits                                                | 13 |
| 12 | Common mistakes                                             | 14 |
| 13 | Exercises                                                   | 15 |
| 14 | Before submitting                                           | 16 |

# 1 Two ways to preserve the past

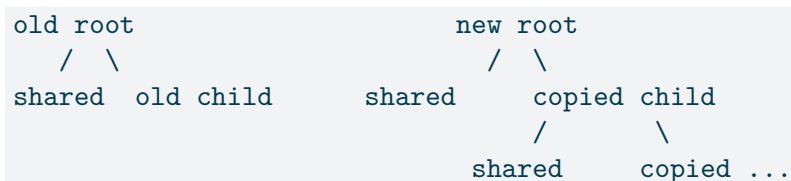
When updates and queries refer to old states, copying the full structure costs too much. Two complementary strategies are common:

- **Persistence:** every update creates a new immutable root and shares all unchanged structure with older versions.
- **Rollback:** mutate one structure while exploring a computation tree, then undo exactly the changes made before returning.

Persistence supports arbitrary branching version queries online. Rollback is ideal when the query order can be organized offline as a DFS.

## 2 Path-copying persistence

A point update in a segment tree touches one root-to-leaf path. Copy those  $O(\log M)$  nodes; every untouched child pointer still refers to the old node.



The invariant is root-specific:

The subtree reached from root  $r$  stores exactly the state of version  $r$  on that node's interval.

Never modify a node reachable from an older root. An update begins by cloning the current node, then recursively replaces only one copied child.

### 3 Persistent lazy propagation

Range updates appear to require pushing a lazy tag into both children, which would copy extra nodes. A cleaner invariant avoids pushing during updates:

```
node.sum = child sums + node.lazy * interval_length
```

The children exclude their parent's tag. A partial query carries the sum of ancestor tags downward. A full-cover query returns `node.sum + carried_tag * length`.

For a partial update, clone the node and affected descendants, recompute its sum with its unchanged local tag, and share every unaffected child. Each update still creates  $O(\log n)$  nodes.

## 4 Rollback DSU

Path compression is excellent for ordinary DSU but destroys the history: one **find** may rewrite many parents. For rollback, use union by size without path compression. Tree height remains logarithmic.

For every successful union, record enough information to undo it:

```
(attached_root, old_size_of_receiving_root)
```

A **snapshot** is the current history length. Roll back by popping records until that length is restored. Failed unions need no record if rollback uses snapshot lengths rather than “undo exactly k calls.”

## 5 Segment tree over time

An added edge is active on a half-open interval  $[\text{start}, \text{end})$ . Insert that edge into the  $O(\log q)$  segment-tree nodes whose time intervals partition its lifetime.

During DFS of the time tree:

1. save a DSU snapshot;
2. unite every edge stored at the current time node;
3. recurse, or answer the operation at a leaf;
4. roll back to the snapshot.

At a leaf for time  $\tau$ , an edge is present in the DSU exactly when its lifetime contains  $\tau$ : exactly one ancestor interval on the root-to-leaf path stores it.

## 6 Cost accounting

| Technique                | Update/preprocessing          | Query       | Memory          |
|--------------------------|-------------------------------|-------------|-----------------|
| persistent point tree    | $O(\log M)$ new nodes         | $O(\log M)$ | $O(q \log M)$   |
| persistent lazy tree     | $O(\log n)$ new nodes         | $O(\log n)$ | $O(n+q \log n)$ |
| time tree + rollback DSU | $O(q \log q \log n)$<br>total | included    | $O(q \log q+n)$ |

## 7 Worked trace: branching versions are a tree, not a timeline

Suppose version zero is empty. Create version 1 by inserting 5, version 2 by inserting 2 into version 1, and version 3 by inserting 8 into version 1.

```
version 0: {}  
  |  
version 1: {5}  
  / \  
version 2: {2,5}    version 3: {5,8}
```

Versions 2 and 3 are siblings. An implementation that updates “the most recent root” cannot represent this input: the update must name its parent root. At the segment-tree level, both siblings share every node not lying on a copied path. They may also share some nodes on their common prefix above the branching point only through version 1; neither sibling may mutate those nodes.

This trace suggests a reliable debugging assertion: after producing a new root, query several older roots again. Their answers must be unchanged.

## 8 Worked trace: why edge lifetimes are half-open

Consider operations at times 0 through 4:

```
0: add 1 2
1: ask 1 2
2: remove 1 2
3: ask 1 2
4: add 1 2
```

The first lifetime is  $[0, 2)$ , not  $[0, 2]$ : the removal operation observes the graph after the edge has disappeared. The second lifetime is  $[4, 5)$ , because an edge still active after the input ends is closed at  $q$ .

When  $[0, 2)$  is decomposed into time-tree nodes, exactly one stored interval lies on the ancestor path of leaf 0 and exactly one on that of leaf 1. None lies on the ancestor path of leaf 2. This is the concrete reason the DSU at a leaf contains exactly the current graph.

## 9 Choosing persistence or rollback

| Input shape                             | Better first model          | Reason                              |
|-----------------------------------------|-----------------------------|-------------------------------------|
| queries name arbitrary old versions     | persistence                 | every root remains queryable        |
| updates form a branching version tree   | persistence or DFS rollback | both expose the branching structure |
| additions and removals on one timeline  | offline rollback            | lifetimes become intervals          |
| online deletions with immediate answers | neither template alone      | future removal times are unknown    |

Do not choose from the data structure being maintained. Choose from the shape of time in the input.

## 10 C++ and Python implementation notes

In C++, reserve the persistent node pool from the operation bound. Indices are usually simpler than raw pointers: index zero can be the all-zero null node, and copying a node is one `push_back(tree[old])`. Use `long long` for stored sums even when updates fit in `int`.

In Python, parallel arrays such as `left`, `right`, and `sum` use much less memory than one object per node. Persistent recursion has logarithmic depth, but a DFS over a path-shaped version tree or time tree can exceed the recursion limit; use an explicit enter/exit stack when the depth can be linear.

# 11 Proof habits

For persistence, prove two statements separately:

1. the new root represents the requested update of its parent version;
2. every old root still represents exactly its old version.

For rollback, define the snapshot precisely as a history length. Prove that undoing to this length restores every mutated field, not merely connectivity. Then prove the traversal invariant: immediately after applying a time-tree node's stored updates, the mutable structure represents all updates attached to the root-to-node path.

## 12 Common mistakes

- Mutating a shared persistent child.
- Pushing a lazy tag without cloning the children being changed.
- Combining node sums and carried tags twice.
- Enabling path compression in a rollback DSU.
- Treating an edge removed at time  $t$  as active at leaf  $t$ .
- Forgetting to close edges still active after the last operation.

## 13 Exercises

A isolates path copying. B forces a precise lazy-sum invariant. C applies rollback to a segment tree over time. D is blind: it asks for an order statistic on a tree path and requires deciding which persistent prefix versions must be added and subtracted.

## 14 Before submitting

Compare against complete copied versions, a fresh BFS graph, or a literal list of values on a reconstructed tree path. Include branching from version zero, negative range additions, repeated add/remove cycles, self-connectivity queries, edges alive until the end, duplicate path values, and  $u=v$ .